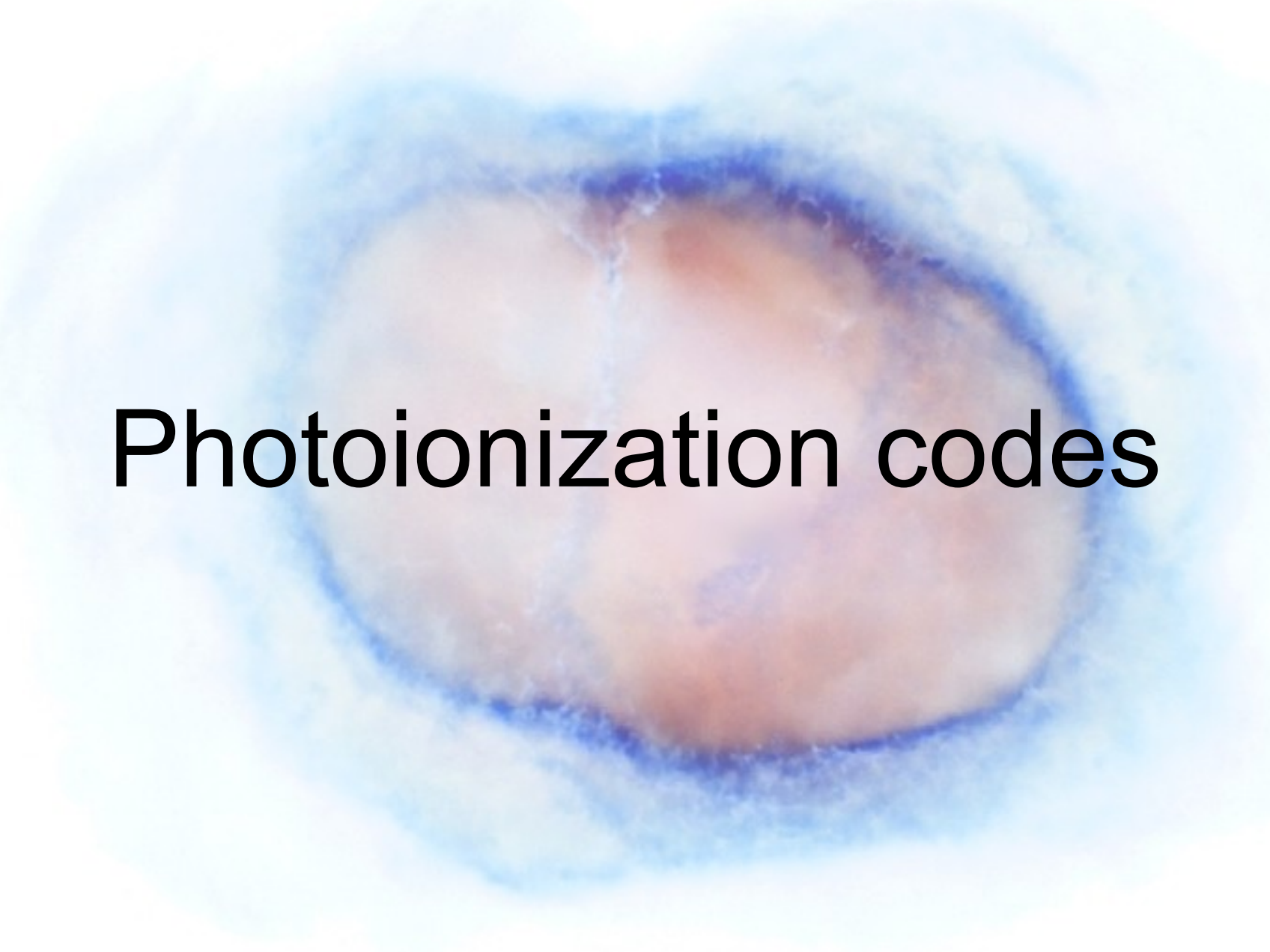




Photoionization codes

Photoionization codes – first ones

- First models in late 60's by D. Flower, P. Harrington, B. Rubin among others.
- Photoionization codes can be seen as a complete summary of all the physics we know about the interaction between energetic radiation and interstellar gas.

COMPUTER MODELS OF THE PLANETARY NEBULAE NGC 7662 AND IC 418

D. R. FLOWER

Department of Physics, University College, London, England

Discrepancies between the observed and calculated spectra of the nebulae are discussed. There is evidence for the importance of dynamical effects and filamentary structure and possibly for deviations of the flux of the central star of NGC 7662 from black-body values in the far ultra-violet.

Photoionization codes - today

- Today the more used photoionization code is certainly **Cloudy** (G. Ferland, P. van Hoof, R. Porter, R. Williams, W. Henney). It's a 200,000 lines C++ code (with 500,000 lines of data files).
- Some other 1D codes are also used (**PHOTO** by Stasinska, **NEBU** by Péquignot, **MAPPINGS** by Dopita and Binette)
- **MOCASSIN** (Ercolano) is a Monte-Carlo full 3D code
- **pyCloudy** (Python version, by Morisset) includes a pseudo-3D code.

Photoionization codes – inputs and outputs

Description of the model (inputs)

- Ionizing SED (BB, stel. model, synth. Spectrum.) T^* , L^* , Z^*
- Gas distribution : $nH(r - x,y,z)$
- Chemical composition (spatial var?)
- Dust (spatial var?)

Atomic database

- Cross sections
- Recomb. coeffs.
- Collision strengths
- Einstein coeffs.
- Dust properties

PHOTOIONISATION CODE

- **Te**, ne
- X^i / X (r - x,y,z)
- Line emissivities

The temperature is an output

- Contrary to the H density, which is an input.
- Results of the thermal equilibrium.
- Obtained by GAINS = LOSS.
- GAINS: depend on ionizing photons mean energy (T^*).

$$G(H^0) = n_e n_p \alpha_A(H^0, T) \frac{\int_{\nu_0}^{\infty} \frac{J_{\nu}}{h\nu} h(\nu - \nu_0) a_{\nu}(H^0) d\nu}{\int_{\nu_0}^{\infty} \frac{J_{\nu}}{h\nu} a_{\nu}(H^0) d\nu}$$

The temperature is an output

- Contrary to the H density, which is an input.
- Results of the thermal equilibrium.
- Obtained by GAINS = LOSS.
- GAINS: depend on ionizing photons mean energy (T^*).
- LOSS: depend on line emission, especially on forbidden lines from metals, thus depend on Z.

$$4\pi j_{ul} = n_u A_{ul} h\nu_{ul} = n_l \frac{g_u}{g_l} e^{-h\nu_{ul}/kT} \left[1 + \left(\frac{A_{ul}}{n_e q_{ul}} \right) \right]^{-1} A_{ul} h\nu_{ul}$$

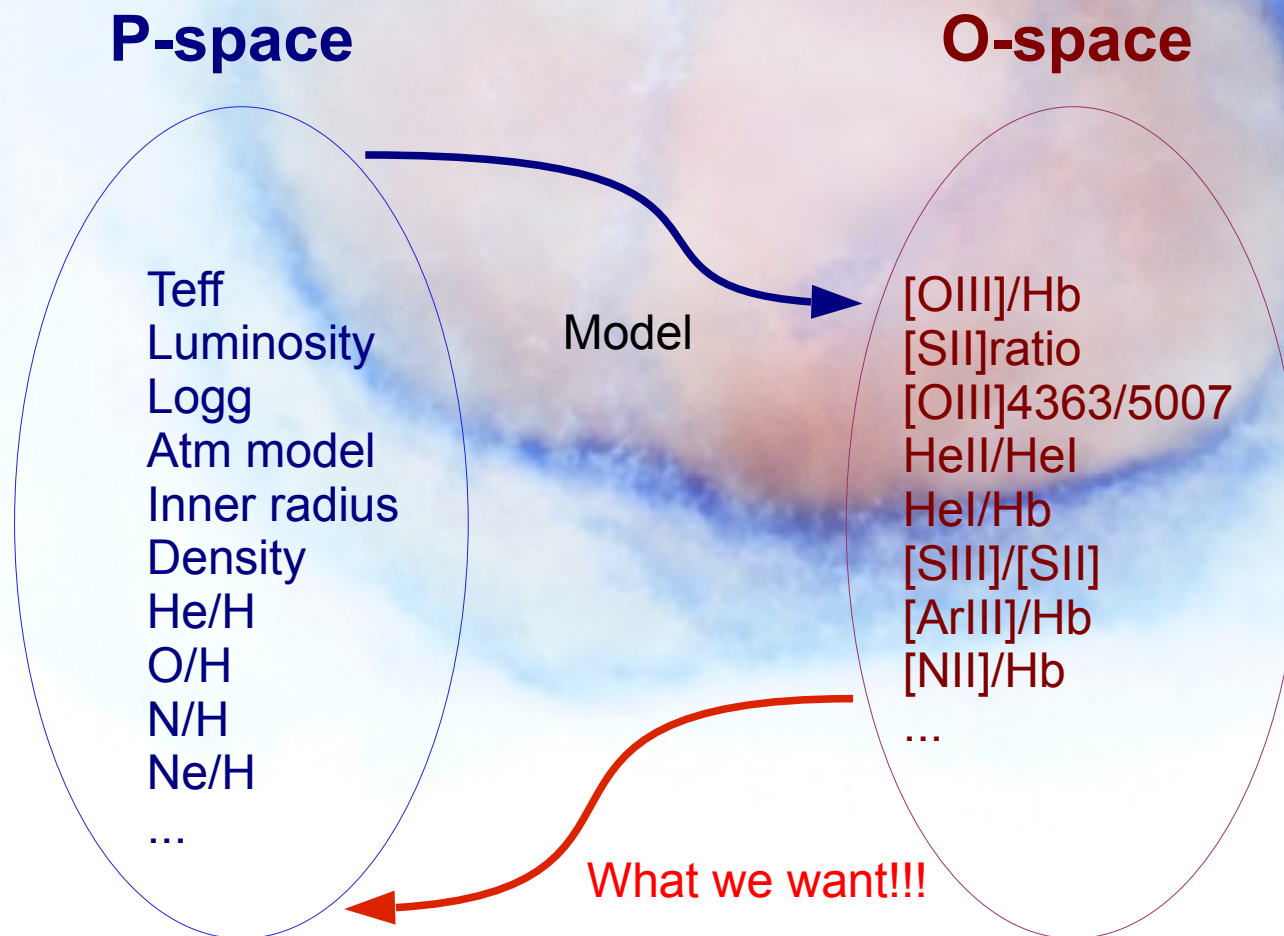
How to make a model ? - basics

Making a model of an given object is **finding** a (unique?) set of **input** parameters that are able to **reproduce all the observables**:

- Emission line ratios (Te- and ne-diagnostics, and also I/I_{beta}).
- Absolute value of one emission line intensity.
- Images (shape and angular size – Distance).
- Continuum flux (radio – IR – Opt – UV – X)
- Line profiles, PV-diagrams (velocity field).

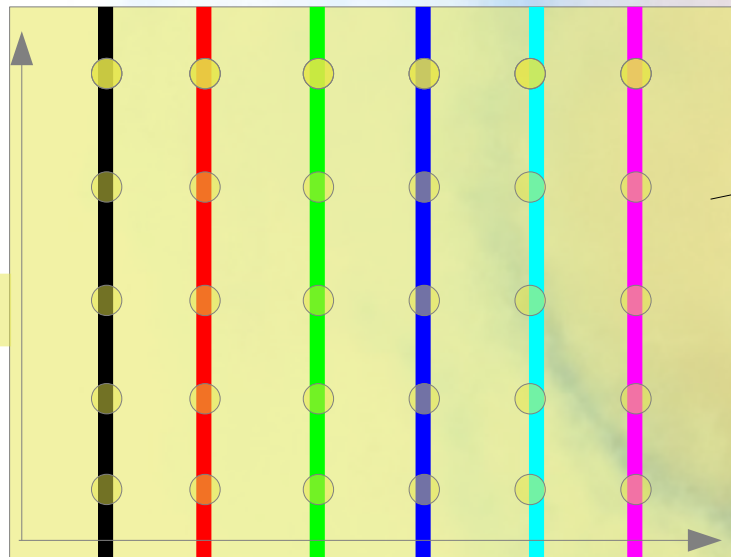
How to make a model ? P- and O-spaces

The modeling process can be seen as solving an inverse problem :



No linearity

P-space



U

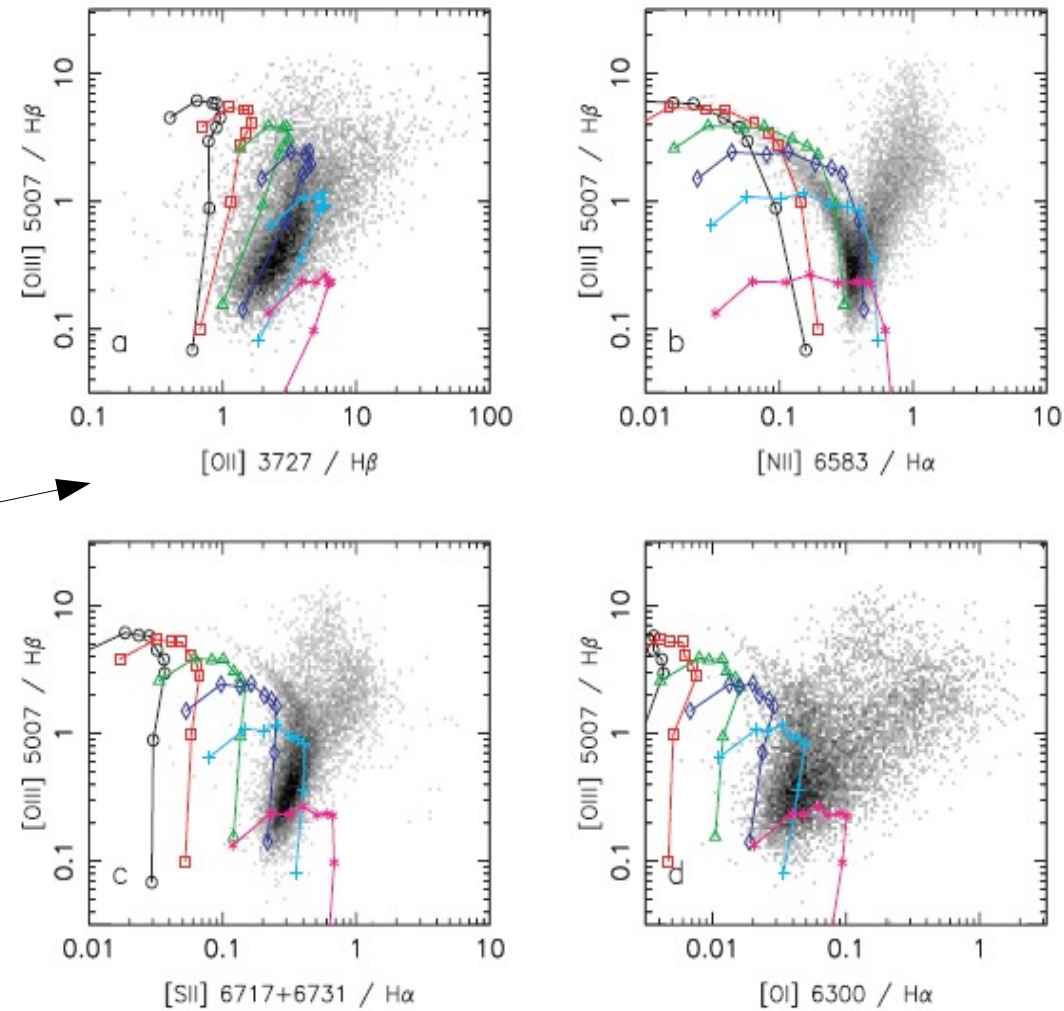


Figure 2. Sequences of photoionization models with varying metallicities Z and constant ionization parameter U . The symbols on the curves correspond to the location of models with metallicities $Z = 0.1, 0.2, 0.3, 0.4, 0.6, 0.8, 1.0, 1.5$ and $2.5 Z_{\odot}$, going from the upper left to the lower right (in panels b, c and d, the lowest metallicity models are actually outside the range of the plots). The values of the ionization parameter U are 10^{-2} (black circles), 5×10^{-3} (red squares), 2×10^{-3} (green triangles), 10^{-3} (blue diamonds), 5×10^{-4} (cyan + signs) and 2×10^{-4} (purple * signs).

Go to Nublado

- <https://www.nublado.org/>
- Contains downloads for the code (current and previous versions), hotfixes, help.
- <https://cloudyastrophysics.groups.io/g/Main> is the forum : report problems, ask for help.
- C23 version of Cloudy is described here :
<https://ui.adsabs.harvard.edu/abs/2023arXiv230806396C>



Short comments on Cloudy/pyCloudy

Cloudy, short how-to

- Cloudy:
 - C++ code (200,000 lines!)
 - freeware
 - easy to install
 - very well supported by an active group of developers/users.
 - New version every 2-3 years (c25.rc1 just released)
 - Very detailed documentation (Quickstart and Hazy)

Cloudy, short how-to

- `Cloudy.exe < model.in > model.out`
- New form : `cloudy.exe -p model`
- Other test.*** files (radius, physics, ionic abundances, emissivities, etc)

Cloudy input file

- Ionizing source:
 - Shape (T[BB], T, Z, logg[Atm Mod], stellar cluster, power law, AGN)
 - Intensity (Luminosity, logU, Q0)
- Gas properties
 - Distribution (R_in, R_out, nH(r), ff)
 - Metallicity
 - Dust

Cloudy standart output

- The standart output from Cloudy is a collection of information about the model without a simple format, described in the Hazy manual.

Look at the output

- Control that everything is OK : check warnings, number of steps, and why it stops (this can be defined in the input file).
- Look at the mean ionization stages of the principal ions, mean temperature, heating and cooling sources.
- Some line intensities.
- More informations are available in other files.

pyStuffs

- pyCloudy
 - Python package to manage Cloudy inputs and outputs,
 - Easy grids,
 - Pseudo-3D facility,
 - Related to PyNeb and 3MdB